

7. Classification Analysis

“Computers are able to see, hear and learn.

Welcome to the future.”

— Dave Waters

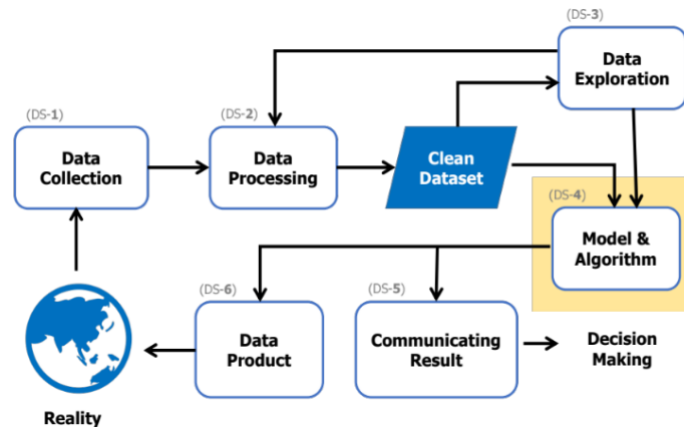


Figure 7.0-1: Classification analysis in the data science process

Many AI applications we encounter daily — facial detection, handwriting recognition, mask-wearing detection, disease screening, automated loan approval — are all built on classification analysis, part of the Model Development (DS-4) stage (Figure 7.0-1). It's an especially valuable, widely applicable technique that every data scientist should master, including how to evaluate and trust the resulting models.

This chapter covers understanding classification analysis, evaluating models, the classification process, key foundational techniques in wide use, and comparing model performance across techniques.

7.1 Understanding Classification Analysis

Classification predicts a label or class. Given a problem, first check whether it has both inputs (features) and an output; if so, it's supervised learning. Then check whether the target is a continuous number or a categorical (nominal-scale) name — the latter signals classification.

Classification data consists of features (inputs, properties, or attributes of a sample) and a class (the labeled group name). In the simple vehicle classification data (Table 2.5-3), weight_kg, height_m, and n_wheels are features, and vtype (car or other) is the class.

Classes fall into two types: Binary Classification (two possible answers, e.g., good/bad, positive/negative, sick/healthy, pass/fail, approved/denied, fraud/no fraud) and Multi-Class Classification (more than two, e.g., blood type A/B/AB/O, grade A/B/C/D/F, vehicle type car/motorcycle/truck/bus, or sentiment positive/neutral/negative). Most classification work is binary, though multi-class problems are also common.

Classification's core job is analyzing features to predict a class, training the model so predictions match true labels as closely as possible — evaluation then decides whether the model is ready for production.

Good classification analysis starts with understanding the problem and data: confirm it's a classification problem, determine binary vs. multi-class (since some techniques handle only one), and check whether class sizes are balanced or skewed — all of which shape data preparation, technique choice, and evaluation method. The following starts with evaluation methods to build intuition, before covering the analysis process and techniques.

7.2 Evaluating Classification Analysis

Classification evaluation centers on accuracy — how correctly items are classified. We'll use the general term “performance” rather than “Accuracy” for the overall concept, since Accuracy is itself one specific, formally defined evaluation method.

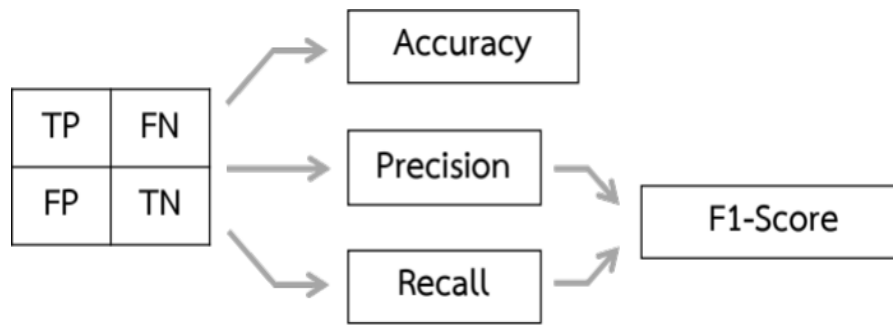


Figure 7.2-1: Sequence of classification evaluation methods

This section covers 5 popular evaluation methods (Figure 7.2-1): Accuracy, Precision, Recall, and F1 (F-Measure) (Igual & Segui, 2017) — all of which start from the Confusion Matrix: True-Positive (TP), True-Negative (TN), False-Positive (FP), and False-Negative (FN). Understanding the confusion matrix is essential to understanding every evaluation method that follows.

Confusion Matrix

For binary classification, the confusion matrix consists of:

- True-Positive (TP) — predicted the desired class, and correct.
- True-Negative (TN) — predicted the undesired class, and correct.
- False-Positive (FP) — predicted the desired class, but wrong.
- False-Negative (FN) — predicted the undesired class, but wrong.

To avoid confusion, “Positive” and “Negative” here simply label the class we do or don't want — not an emotional judgment. “Positive” labels whatever the model predicts as the wanted class; “Negative” labels the other. For example:

1. Building a model to select good fruit: the wanted class is good fruit, so a “good” prediction is labeled Positive, and “not good” is Negative.
2. Building a model to count cars: the wanted class is car, so a “car” prediction is Positive, and other vehicle types are Negative.
3. Building a model to screen patients: the wanted class is patient, so a “patient” prediction is Positive, and “not a patient” is Negative.
4. Building a model to detect burglars: the wanted class is burglar, so a “burglar” prediction is Positive, and “not a burglar” is Negative.

Even though “patient” and “burglar” carry negative connotations in everyday language, they're still labeled “Positive” here, since that's simply the class of interest — don't let emotional associations influence this technical labeling.

Once the model predicts Positive or Negative, check against the truth: a correct guess is True, a wrong guess is False.

- Predicted Positive, actually Positive → True-Positive (TP).
- Predicted Negative, actually Negative → True-Negative (TN).
- Predicted Positive, actually Negative → False-Positive (FP).
- Predicted Negative, actually Positive → False-Negative (FN).

For the car-counting example, where a “car” prediction is Positive:

- A real car counted as a car — True-Positive (TP).
- A non-car not counted as a car — True-Negative (TN).
- A non-car counted as a car — False-Positive (FP).
- A real car not counted as a car — False-Negative (FN).

These counts are recorded in a confusion matrix (Table 7.2-1), with rows as Actual class and columns as Predicted class:

Table 7.2-1: The confusion matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

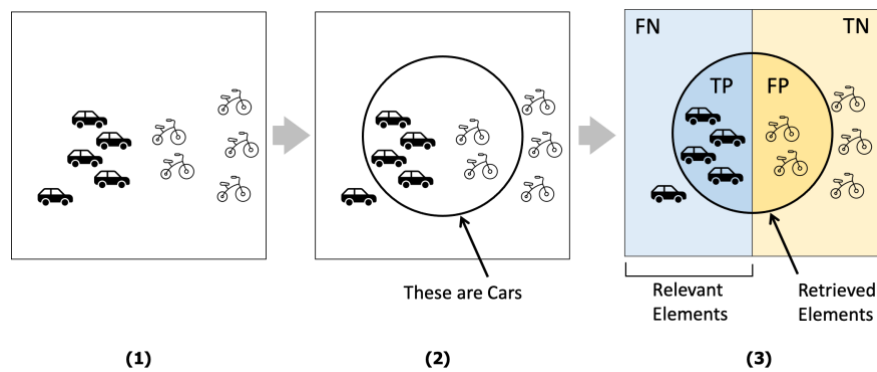


Figure 7.2-2: Illustrating the confusion matrix with an example

For example (Figure 7.2-2): 5 real cars and 5 other vehicles (panel 1). The model selects 6 objects it predicts are cars (panel 2, circled "These are cars"), leaving 4 objects predicted as non-cars. This breaks down as (panel 3):

- 5 objects on the left are the Relevant Elements — the true target class (cars).
- The model selected 6 Retrieved Elements (circled) — predicted as the car class (Positive).
- Of those 6, 4 are actually cars — correct → True-Positive (TP) = 4.
- Of those 6, 2 are not actually cars — wrong → False-Positive (FP) = 2.
- The remaining 4 unselected objects are predicted non-cars (Negative).
- Of those 4, 3 are genuinely non-cars — correct → True-Negative (TN) = 3.
- Of those 4, 1 is actually a car — wrong → False-Negative (FN) = 1.

In practice, data is usually tabular, so the evaluation table must include each sample's true class, as in Table 7.2-2 — the same car-counting example, with 10 cars total, 6 predicted as cars, 5 truly cars, and 5 other.

Table 7.2-2: Example predictions from a car-counting system

Car #	Actual Type (y)	Predicted Type ($\hat{y}$)	Confusion Matrix
1	car	car	TP
2	car	car	TP
3	car	car	TP
4	car	car	TP
5	car	not car	FN
6	other	car	FP
7	other	car	FP
8	other	not car	TN

Car #	Actual Type (y)	Predicted Type (ŷ)	Confusion Matrix
9	other	not car	TN
10	other	not car	TN

Table 7.2-3: Resulting confusion matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP = 4	FN = 1
	Negative	FP = 2	TN = 3

In practice, a good model should have high TP and TN, with FP and FN close to zero. But comparing 4 values across models is unwieldy, so the following single-value measures are derived from them.

Accuracy

Accuracy is the most direct measure: the proportion of correctly classified items (TP and TN) out of all predictions. It suits situations where Positive and Negative classes are similarly sized.

Equation

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy ranges 0 to 1 — closer to 1 is better, closer to 0 is worse. Comparing models, prefer the one with higher Accuracy.

Worked Example

From Table 7.2-3:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{4 + 3}{4 + 3 + 2 + 1} = \frac{7}{10} = 0.7$$

Code Example

```
from sklearn.metrics import accuracy_score

y_test = ['car', 'car', 'car', 'car', 'car',
          'other', 'other', 'other', 'other', 'other']
y_pred = ['car', 'car', 'car', 'car', 'other',
          'car', 'car', 'other', 'other', 'other']
print("Accuracy:", accuracy_score(y_test, y_pred))

Accuracy: 0.7
```

Limitation

Accuracy suits balanced classes, but breaks down with imbalanced ones. Suppose 100 cars include only 3 real cars, with 97 other vehicles; if the model predicts “not a car” for all 100, giving TP=0, TN=97, FP=0, FN=3:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{0 + 97}{0 + 97 + 0 + 3} = \frac{97}{100} = 0.97$$

This gives a very high Accuracy despite the model never correctly identifying a single car — useless in practice. This happens with Imbalanced Classes, common in anomaly-detection problems (burglary, spam

detection), where the rare positive class gets swamped by an abundant negative class, inflating TP+TN via a large TN term.

So: use Accuracy when class sizes are similar; when they differ greatly, prefer Precision, Recall, or F1 instead, which don't rely on TN.

Precision

Precision measures how much of what the model selected actually matches what's wanted — as in Figure 7.2-3, sorting diamonds from rocks: the proportion of correct Positive predictions (TP) out of everything predicted Positive (TP+FP).

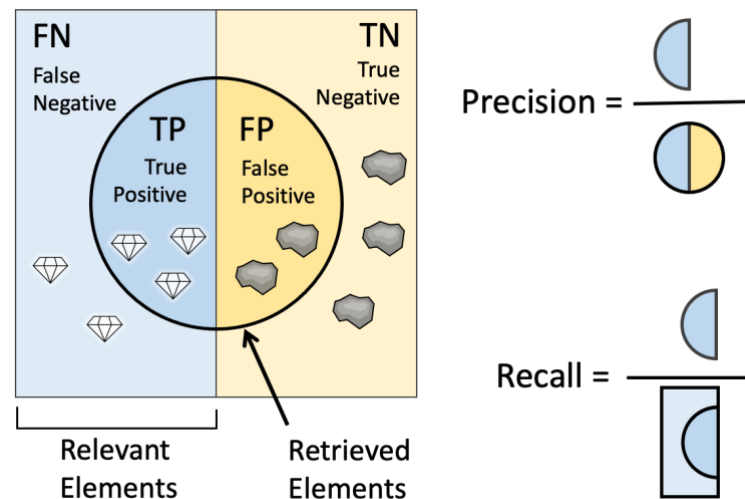


Figure 7.2-3: Illustrating Precision and Recall

Equation

$$Precision = \frac{TP}{TP + FP}$$

Precision ranges 0 to 1, higher (closer to 1) is better.

Worked Example

$$Precision = \frac{TP}{TP + FP} = \frac{4}{4 + 2} = \frac{4}{6} = 0.67$$

Code Example

```
from sklearn.metrics import precision_score

y_test = ['car', 'car', 'car', 'car', 'car',
          'other', 'other', 'other', 'other', 'other']
y_pred = ['car', 'car', 'car', 'car', 'other',
          'car', 'car', 'other', 'other', 'other']
print("Precision:", precision_score(y_test, y_pred, pos_label="car"))

Precision: 0.6666666666666666
```

Limitation

Precision uses only TP and FP — examples the model labeled Positive. So a model that labels very few things Positive, but gets them right, scores a deceptively high Precision. E.g., 100 vehicles include 90 real cars; the model predicts only 2 as Positive, both correct, giving TP=2, TN=10, FP=0, FN=88:

$$Precision = \frac{TP}{TP + FP} = \frac{2}{2 + 0} = \frac{2}{2} = 1.0$$

A perfect Precision score, yet the confusion matrix shows this model can't be trusted — it barely selects anything. Precision alone is often insufficient; in practice, pair it with Recall, covered next.

Recall

Recall measures how much of what's actually wanted got selected by the model (Figure 7.2-3) — the proportion of correctly predicted Positives (TP) out of all truly Positive cases (TP+FN).

Equation

$$Recall = \frac{TP}{TP + FN}$$

Recall ranges 0 to 1, higher (closer to 1) is better.

Worked Example

$$Recall = \frac{TP}{TP + FN} = \frac{4}{4 + 1} = \frac{4}{5} = 0.8$$

Code Example

```
from sklearn.metrics import recall_score

y_test = ['car', 'car', 'car', 'car', 'car',
          'other', 'other', 'other', 'other', 'other']
y_pred = ['car', 'car', 'car', 'car', 'other',
          'car', 'car', 'other', 'other', 'other']
print("Recall:", recall_score(y_test, y_pred, pos_label="car"))

Recall: 0.8
```

Limitation

If a model predicts every example as the Positive class, giving TP=5, TN=0, FP=5, FN=0:

$$Recall = \frac{TP}{TP + FN} = \frac{5}{5 + 0} = \frac{5}{5} = 1.0$$

A perfect Recall score, but the confusion matrix again shows this model can't be trusted, despite the high Recall.

F1 (or F-Measure)

Since Precision and Recall together give a fuller picture for imbalanced classes, but a single comparable value is often preferred, F1 was developed as the harmonic mean of Precision and Recall.

Equation

Derived from the harmonic-mean definition:

$$\frac{1}{F1} = \frac{1}{Precision} + \frac{1}{Recall}$$

Rearranging gives the usable form:

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1 ranges 0 to 1, higher (closer to 1) is better.

Worked Example

With Precision = 0.67 and Recall = 0.8:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = 2 \times \frac{0.67 \times 0.8}{0.67 + 0.8} = 2 \times \frac{0.54}{1.47} = 0.73$$

Code Example

```
from sklearn.metrics import f1_score

y_test = ['car', 'car', 'car', 'car', 'car',
          'other', 'other', 'other', 'other', 'other']
y_pred = ['car', 'car', 'car', 'car', 'other',
          'car', 'car', 'other', 'other', 'other']
print("F1:", f1_score(y_test, y_pred, pos_label="car"))

F1: 0.7272727272727272
```

Practical Use

Precision and Recall use TP, FP, and FN, deliberately avoiding TN, which causes problems for imbalanced classes. In practice: use Accuracy when both classes are similarly sized; use F1 (F-Measure) when sizes differ greatly.

Even with F1 as a single value, research work still benefits from examining Precision and Recall separately, to understand which to prioritize improving.

For multi-class problems, even with equal group sizes (e.g., classes A, B, C, D with 5 members each, 20 total), the effective comparison is always “class A vs. not,” “class B vs. not,” and so on — each becomes a 5-vs-15 imbalanced comparison. So multi-class problems should generally use F1 rather than Accuracy.

7.3 The Classification Analysis Process

The classification process mirrors regression, following the model-building process from section 5.4.2:

1. Import Data — structured, ready-to-use data.
2. Basic Data Processing — handle missing data, outliers, or unsuitable structure; skip if already done.
3. Check Data Behavior — data type, range, and distribution per feature; skip if already done.
4. Run Experiments for Each Experimental Environment, with sub-steps:
 5. 4.1 Define the experimental environment — choosing a classification technique, its parameter arguments, and features.
 6. 4.2 Build and test the model (multiple rounds via K-Fold Cross-Validation, or one round via Train-Test Split): split data (X_train/y_train, X_test/y_test); transform features if needed; build a temporary model; predict on X_test to get y_pred; evaluate against y_test.
 7. 4.3 Summarize this environment's results — average across rounds.
8. Compare Results Across Experiments.
9. Build the Final Model from the chosen environment, using the entire dataset.
10. Deploy the Model.

As before: to update the model with new data using the same environment, redo steps 6–7 with the latest data; to search for a better environment, start over from step 1.

7.4 Classification Techniques

This section covers 5 foundational, widely used techniques (Bishop & Nasrabadi, 2006): Logistic Regression, Decision Tree, Naïve Bayes, K-Nearest Neighbor, and Artificial Neural Network. Each covers its principle (concept, equation, worked example), usage guidance (especially around feature data types), and a Python code example using Pandas and Scikit-Learn, following the process from section 7.3, focused on building, predicting, and evaluating.

Logistic Regression

A binary classification technique using the logistic regression equation to turn a set of features into a class 0 or 1 output. It's simple, popular, and easy to understand.

Principle

$$P = \frac{1}{1 + e^{-(b_0 + BX)}}$$

- P — the probability the predicted example belongs to class 0 or 1
- X — the vector of feature values for the sample: $x_1, x_2, x_3, \dots, x_n$
- B — the vector of coefficients for each feature in X : $b_1, b_2, b_3, \dots, b_n$
- b_0 — the y-intercept constant

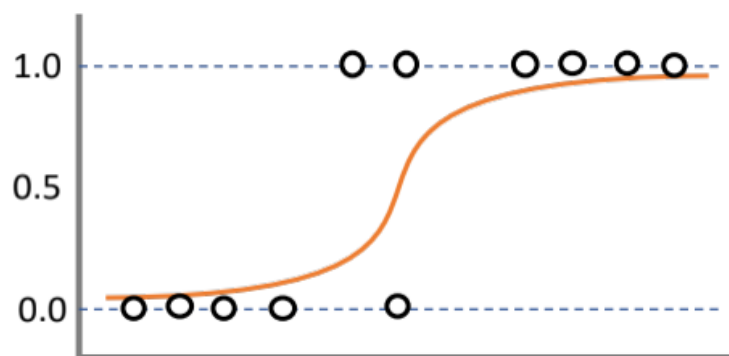


Figure 7.4-1: Graph of the logistic regression equation

Since $x_1, x_2, \dots, x_n$ are numeric, this works directly with interval- or ratio-scale data.

Example: with 2 features, learned coefficients $b_0=1, b_1=2, b_2=3$:

$$P = \frac{1}{1 + e^{-(1+2x_1+3x_2)}}$$

For a sample with $x_1=4, x_2=5$:

$$P = \frac{1}{1 + e^{-(1+2x_1+3x_2)}} = \frac{1}{1 + e^{-(1+2(4)+3(5))}} = \frac{1}{1 + e^{-24}} \approx 1 \rightarrow \text{class 1}$$

For a sample with $x_1=-4, x_2=-5$:

$$P = \frac{1}{1 + e^{-(1+2x_1+3x_2)}} = \frac{1}{1 + e^{-(1+2(-4)+3(-5))}} = \frac{1}{1 + e^{24}} \approx 0 \rightarrow \text{class 0}$$

Usage Guidance

Works directly with interval- or ratio-scale features. Ordinal data should be encoded as ordered numbers; nominal data with only 2 values can be encoded 0/1 directly; with 3+ values, use One-Hot Encoding first.

Code Example

Using the simple vehicle classification data (section 2.5.1):

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

csv_path = "https://rathachai.github.io/DA-LAB/datasets/simple-veh-
class.csv"
df = pd.read_csv(csv_path)
X = df[['weight_kg', 'height_m', 'n_wheels']]
y = df['vtype']
model = LogisticRegression()
model.fit(X,y)
```

```
print("classes", model.classes_)
print("features:", X.columns.tolist())
print("coef:", model.coef_)
print("intercept:", model.intercept_)

classes ['car' 'other']
features: ['weight_kg', 'height_m', 'n_wheels']
coef: [[2.51388865e-04 3.31737815e-01 7.66911472e-01]]
intercept: [-4.12665239]
```

```
new_X = [(1000,1,4), (2000,2,6)]
y_pred = model.predict(new_X)
print("results: ", y_pred)

results:  ['car' 'other']
```

Decision Tree

An intuitive, popular technique that builds a hierarchy of conditional rules, branching repeatedly until reaching a leaf that gives the decision.

Principle

A decision tree has nodes, branches, and leaves: a node specifies a feature to check, branches represent that feature's condition outcomes leading to the next node, and leaves indicate the resulting class. Figure 7.4-2 illustrates this for deciding whether to watch a movie with a friend.

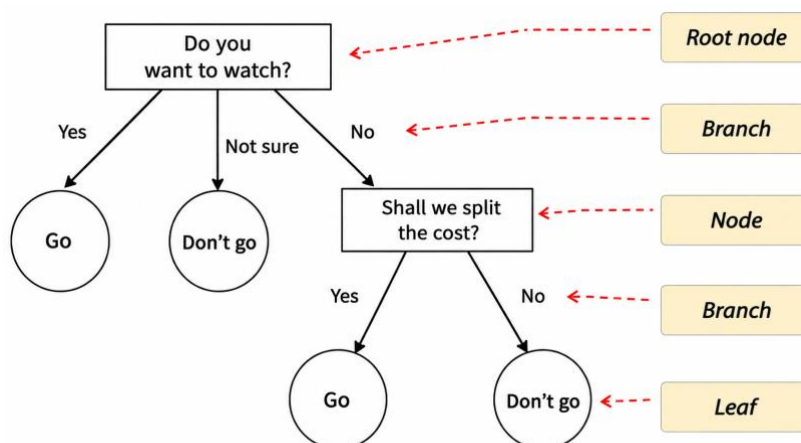


Figure 7.4-2: Example decision tree model

Experts can build decision trees manually, but in data-driven practice, machines build them from data using an algorithm such as Iterative Dichotomiser 3 (ID3), using an impurity measure — Entropy or the GINI Coefficient — to decide which feature becomes the root node and subsequent nodes, splitting until each branch reaches a single (or dominant) class, becoming a leaf.

This book covers the GINI Coefficient, borrowed from economics as an inequality measure:

$$GINI = 1 - \sum_{c \in C} \left(\frac{|S_c|}{|S|} \right)^2$$

- c — a class, member of the full class set C
- $|S|$ — the number of examples split into a given branch by a feature condition
- $|S_c|$ — the number of those examples belonging to class c

GINI ranges 0 to 1: 0 means a branch contains only one class; more mixing pushes it toward 1. The best splitting feature minimizes GINI.

Example: splitting data into 5 examples — 3 car, 2 other — gives $GINI = 0.48$:

$$GINI = 1 - \sum_{c \in C} \left(\frac{|S_c|}{|S|} \right)^2 = 1 - \left(\frac{|S_{car}|}{|S|} \right)^2 - \left(\frac{|S_{other}|}{|S|} \right)^2 = 1 - \left(\frac{3}{5} \right)^2 - \left(\frac{2}{5} \right)^2 = 0.48$$

Usage Guidance

Since ID3 splits by feature value using an impurity measure, it strictly needs nominal- or ordinal-scale data; interval/ratio data must first be converted. However, flexible tools like Scikit-Learn support all 4 data scales directly.

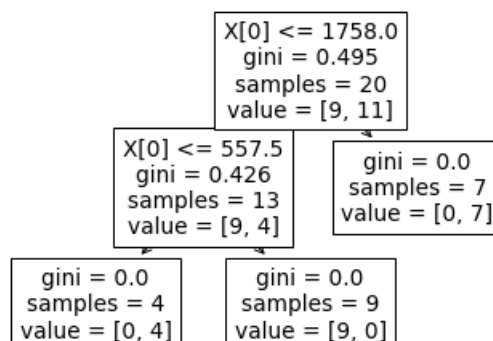
Code Example

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

csv_path = "https://rathachai.github.io/DA-LAB/datasets/simple-veh-
class.csv"
df = pd.read_csv(csv_path)
X = df[['weight_kg', 'height_m', 'n_wheels']]
y = df['vtype']
model = DecisionTreeClassifier()
model.fit(X, y)
```

Viewing the tree's structure:

```
from sklearn.tree import plot_tree
plot_tree(model)
```



This can be interpreted as: the root node splits on $X[0]$ (weight_kg) with condition $X[0] \leq 1758.0$, giving $GINI=0.495$ across 20 samples (value=[9, 11] — 9 car, 11 other, per `model.classes_`). If true ($\text{weight} \leq 1758.0$), it proceeds to check $X[0] \leq 557.5$: true leads to a leaf predicting other; false leads to a leaf predicting car. If the root condition is false ($\text{weight} > 1758.0$), it leads directly to a leaf predicting other.

```
new_X = [(1000, 1, 4), (2000, 2, 6)]
y_pred = model.predict(new_X)
print("results: ", y_pred)

results:  ['car' 'other']
```

Naïve Bayes

Named after British statistician Thomas Bayes, this technique uses conditional probability between a later event and an earlier one, to predict one event's likelihood from the other.

Principle

This popular technique applies probability theory between class and feature, using a sample's feature values to estimate class probability:

$$P(c|a) = \frac{P(a|c) \times P(c)}{P(a)}$$

- $P(c|a)$ — read “the probability of c given a ”: the probability this sample belongs to class c , given its feature value a
- $P(a|c)$ — the probability of feature value a , given class c
- $P(c)$ — the probability of class c occurring
- $P(a)$ — the probability of feature value a occurring

The class with the highest $P(c|a)$ across all classes is chosen:

$$class = \arg \max_{c \in C} P(c|a)$$

With multiple features $a_1, a_2, \dots, a_n$, the formula extends accordingly. Since $P(a_1, a_2, \dots, a_n|c)$ assumes conditional independence, it factors into a product:

$$P(c|a_1, a_2, \dots, a_n) = \frac{P(a_1, a_2, \dots, a_n|c) \times P(c)}{P(a_1, a_2, \dots, a_n)}$$

Since $P(a_1, a_2, \dots, a_n)$ is constant across all classes being compared, it can be dropped, giving the final classification rule:

$$P(a_1, a_2, \dots, a_n|c) = P(a_1|c) \times P(a_2|c) \times \dots \times P(a_n|c) = \prod_i P(a_i|c)$$

In practice, if any $P(a_i|c) = 0$, the whole product becomes 0; add-one smoothing (adding 1 to numerator and denominator) avoids this.

Worked example, using the flu diagnosis dataset (section 2.5.3): given normal temperature, headache=yes, and nausea=yes, is this flu (flu=yes) or not (flu=no)? Using add-one smoothing throughout, Table 7.4-1 shows each term:

Table 7.4-1: Example Naïve Bayes probability calculations

Term	Concept	Result
$P(\text{flu}=\text{yes})$	cases with flu ÷ total cases	$(3+1) \div (7+1) = 0.5$
$P(\text{flu}=\text{no})$	cases without flu ÷ total cases	$(4+1) \div (7+1) = 0.63$
$P(\text{temp}=\text{normal} \text{flu}=\text{yes})$	normal-temp & flu cases ÷ flu cases	$(0+1) \div (3+1) = 0.25$
$P(\text{temp}=\text{normal} \text{flu}=\text{no})$	normal-temp & no-flu cases ÷ no-flu cases	$(3+1) \div (4+1) = 0.8$
$P(\text{headache}=\text{yes} \text{flu}=\text{yes})$	headache & flu cases ÷ flu cases	$(3+1) \div (3+1) = 1$
$P(\text{headache}=\text{yes} \text{flu}=\text{no})$	headache & no-flu cases ÷ no-flu cases	$(1+1) \div (4+1) = 0.4$
$P(\text{nausea}=\text{yes} \text{flu}=\text{yes})$	nausea & flu cases ÷ flu cases	$(2+1) \div (3+1) = 0.75$
$P(\text{nausea}=\text{yes} \text{flu}=\text{no})$	nausea & no-flu cases ÷ no-flu cases	$(2+1) \div (4+1) = 0.6$

```

P (flu=yes | temp=normal, headache=yes, nausea=yes)
= P(flu=yes) P(temp=normal| flu=yes) P(headache=yes| flu=yes)
P(nausea=yes| flu=yes)
= (0.5) (0.25) (1) (0.75)
= 0.09
P (flu=no | temp=normal, headache=yes, nausea=yes)
= P(flu=no) P(temp=normal| flu=no) P(headache=yes| flu=no)
P(nausea=yes| flu=no)
= (0.63) (0.8) (0.4) (0.6)
= 0.12

```

Since $P(\text{flu}=\text{no}|\dots)$ is higher, this case is classified as flu=no.

Usage Guidance

Since Naïve Bayes computes probabilities per feature value, it strictly needs nominal- or ordinal-scale data; interval/ratio data must first be binned. Features should also be independent of each other. Scikit-Learn also supports continuous numeric inputs directly.

Code Example

Using Scikit-Learn, categorical values must first be encoded as numbers via LabelEncoder (e.g., high, very_high, normal $\rightarrow$ 0, 1, 2), using the flu diagnosis dataset (section 2.5.3):

```

import pandas as pd

csv_path = "https://rathachai.github.io/DA-LAB/datasets/flu-test.csv"
df = pd.read_csv(csv_path)

```

```

from sklearn.preprocessing import LabelEncoder

temp_encoder = LabelEncoder()
temp_encoder.fit(df['temp'])
headache_encoder = LabelEncoder()

```

```
headache_encoder.fit(df['headache'])
nausea_encoder = LabelEncoder()
nausea_encoder.fit(df['nausea'])
```

```
df["temp_code"] = temp_encoder.transform(df['temp'])
df["headache_code"] = headache_encoder.transform(df['headache'])
df["nausea_code"] = nausea_encoder.transform(df['nausea'])
X = df[['temp_code', 'headache_code', 'nausea_code']]
y = df['flu']
print(X)
```

	temp_code	headache_code	nausea_code
0	0	0	1
1	2	0	0
2	1	1	1
3	0	0	0
4	0	1	0
5	1	0	1
6	1	1	0

```
from sklearn.naive_bayes import CategoricalNB

model = CategoricalNB()
model.fit(X,y)
```

```
temp_code = temp_encoder.transform(["normal"])[0]
headache_code = headache_encoder.transform(["YES"])[0]
nausea_code = nausea_encoder.transform(["YES"])[0]
new_X = [ (temp_code, headache_code, nausea_code) ]
y_pred = model.predict(new_X)
print("new_X :", new_X)
print("results: ", y_pred)

new_X : [(1, 0, 0)]
results: ['no']
```

K-Nearest Neighbor

Classifies a new sample by checking which class dominates among its nearest neighbors.

Principle

Analysis starts by computing the distance between the sample to classify and every labeled sample, using their feature values. The K closest examples become its Nearest Neighbors; the sample takes on whichever class is most common among them.

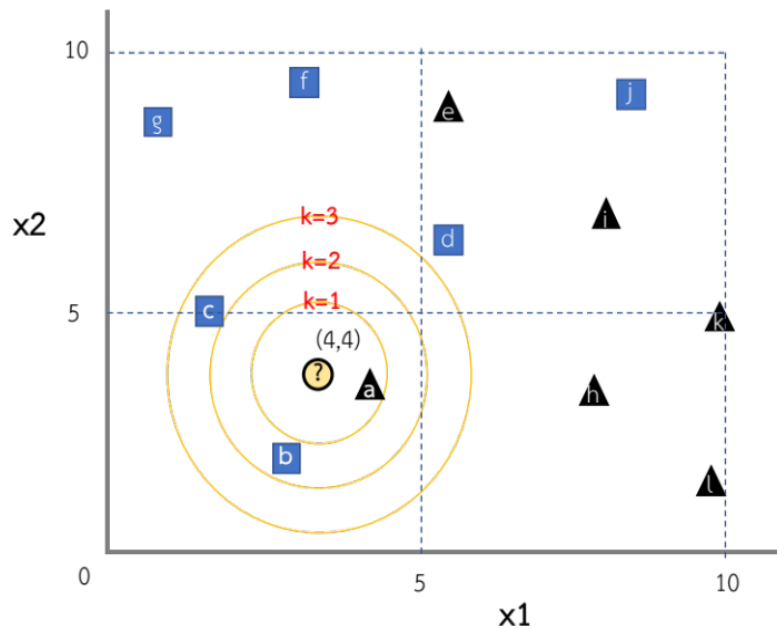


Figure 7.4-3: Example of K-Nearest Neighbor in action

Example: a dataset with 2 classes (triangle, square) and 2 features X_1 , X_2 . A new sample at (4,4) has neighbors ordered nearest-to-farthest: a, b, c, d, e, f, g, h, i, j, k, l. Classifying with different K :

- $K=1$: nearest neighbor is a (triangle) → classified as triangle.
- $K=2$: nearest neighbors are a, b (1 triangle, 1 square) → tied, can't decide.
- $K=3$: nearest neighbors are a, b, c (1 triangle, 2 square) → classified as square.

In practice, choose K carefully — an odd number avoids ties, and not too small, to avoid being thrown off by noise.

Usage Guidance

Since this relies on distance, features must be numeric — works directly with interval- or ratio-scale data; ordinal or nominal data must be converted to numbers first.

Code Example

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

csv_path = "https://rathachai.github.io/DA-LAB/datasets/simple-veh-
class.csv"
df = pd.read_csv(csv_path)
X = df[['weight_kg', 'height_m', 'n_wheels']]
y = df['vtype']
model = KNeighborsClassifier()
model.fit(X,y)
new_X = [(1000,1,4), (2000,2,6)]
y_pred = model.predict(new_X)
print("results: ", y_pred)

results:  ['car' 'other']
```

Artificial Neural Network

Perhaps the most widely discussed AI/machine-learning technique today, abbreviated ANN or simply Neural Network, simulating networks of connected neurons for pattern recognition and prediction — also the foundation for deep learning's more advanced techniques. This book covers ANN's core principles as a foundation for further study.

Principle

An ANN connects many neurons, each called a node or perceptron — the smallest unit, taking inputs and producing an output passed to other nodes, structured as in Figure 7.4-4.

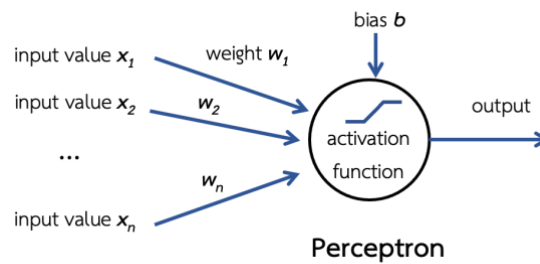


Figure 7.4-4: Structure of a perceptron

Each perceptron takes inputs $x_1, x_2, \dots, x_n$, each with an associated weight ($w_1, w_2, \dots, w_n$) multiplying it, plus a bias term. Weights and bias are coefficients determined through learning from data. The weighted sum then passes through an activation function to produce the output. Common activation functions include Step, Sigmoid (Logistic), tanh, and ReLU (Rectified Linear Unit), shown in Figure 7.4-5 with their graphs and equations.

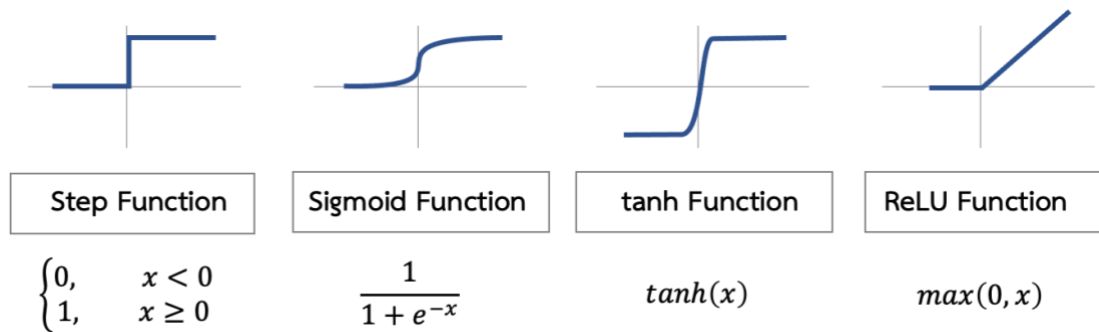


Figure 7.4-5: Example activation functions

The output formula, where f is any activation function, w_i is the weight at position i , x_i is the input at position i (of n total inputs), and b is the bias:

$$output = f\left(b + \sum_{i=1}^n w_i x_i\right)$$

Example: a perceptron using the Sigmoid function (sig), with inputs 0.1 and 0.2, weights 0.3 and 0.4, and bias -0.01:

$$\begin{aligned} output &= sig\left(b + \sum_{i=1}^n w_i x_i\right) \\ &= sig(-0.01 + (0.1 \times 0.3 + 0.2 \times 0.4)) \\ &= sig(0.1) = \frac{1}{1 + e^{-0.1}} = 0.52 \end{aligned}$$

Multiple perceptrons combine into a network structured in layers (Figure 7.4-6):

- Features — the input data from the dataset.
- Input layer — receives features directly; a single layer, with one node per feature.

- Hidden layer(s) — internal processing layers, each with multiple nodes, connected by weighted edges passing data to the next layer.
- Output layer — the final layer, processing the last hidden layer's output; its node count matches the number of classes.

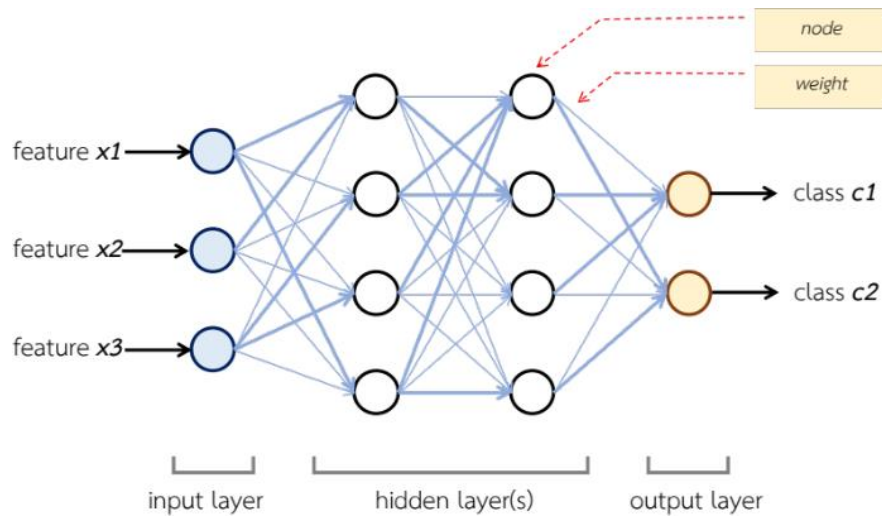


Figure 7.4-6: Structure of an artificial neural network

Developers must specify the number of hidden layers, nodes per layer, and each layer's activation function. Training tools randomly initialize weights, test predictions against actual values, and iteratively adjust weights until predictions are as accurate as possible.

Usage Guidance

Since perceptron inputs must be numeric, this works directly with interval- or ratio-scale data; ordinal or nominal data must be converted to numbers first.

Developers should experiment with the number of hidden layers, nodes per layer, and activation functions for best performance — starting with a single hidden layer sized between the number of features and number of classes, then adjusting from there.

Be aware that too many hidden layers increases training time, and risks the model memorizing inputs and outputs rather than learning genuine patterns (Overfitting) — fitting the training data too closely to generalize well, and performing poorly on new data despite excellent training performance.

Code Example

Scikit-Learn's ANN implementation is called `MLPClassifier` (Multi-Layer Perceptron):

```
import pandas as pd
from sklearn.neural_network import MLPClassifier

csv_path = "https://rathachai.github.io/DA-LAB/datasets/simple-veh-
class.csv"
df = pd.read_csv(csv_path)
X = df[['weight_kg', 'height_m', 'n_wheels']]
y = df['vtype']
model = MLPClassifier(hidden_layer_sizes=(15), activation='tanh')
model.fit(X, y)
new_X = [(1000, 1, 4), (2000, 2, 6)]
y_pred = model.predict(new_X)
print("results: ", y_pred)

results:  ['car' 'other']
```

7.5 Comparing Model Performance

This section compares the 5 techniques on the simple vehicle classification data (section 2.5), using Logistic Regression, Decision Tree, Naïve Bayes, K-Nearest Neighbor, and Artificial Neural Network, tested via 4-Fold Cross-Validation and evaluated with Accuracy (since classes are balanced here), following the process from section 7.3.

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score
```

Step 1 (Import Data) and Step 2 (Basic Data Processing) — Step 3 is skipped in this example:

```
# step: 1
csv_path = "https://rathachai.github.io/DA-LAB/datasets/simple-veh-
class.csv"
df = pd.read_csv(csv_path)
# step: 2
X = df[['weight_kg', 'height_m', 'n_wheels']]
y = df['vtype']
# step: 3 (n/a)
```

Step 4.1 — defining each experimental environment: Logistic Regression, Decision Tree, Naïve Bayes, K-Nearest Neighbor (k=3 and k=5), and Artificial Neural Network (1 hidden layer with 10 or 100 nodes):

```
# step: 4
# step: 4.1
envs={}
envs["LR"] = LogisticRegression()
envs["DT"] = DecisionTreeClassifier()
envs["NB"] = GaussianNB()
envs["KNN (k3)"] = KNeighborsClassifier(n_neighbors=3)
envs["KNN (k5)"] = KNeighborsClassifier(n_neighbors=5)
envs["ANN (h10)"] = MLPClassifier(hidden_layer_sizes=(10,),
    activation='logistic', max_iter=5000)
envs["ANN (h100)"] = MLPClassifier(hidden_layer_sizes=(100,),
    activation='logistic', max_iter=5000)
env_results = {} # for storing results
```

Step 4.2 (build and test each environment via 4-Fold Cross-Validation) and Step 4.3 (average results per environment):

```
for env_name in envs.keys():
    acc_list = [] # for storing accuracy in each fold
    # step: 4.2
    kf = KFold(n_splits=4)
    for train_index, test_index in kf.split(X):
        # step: 4.2.1
        X_train, X_test = X.loc[train_index], X.loc[test_index]
        y_train, y_test = y[train_index], y[test_index]
        # step: 4.2.2 (n/a)
        # step: 4.2.3
        model = envs[env_name].fit(X_train, y_train)
```

```

# step: 4.2.4
y_pred = model.predict(X_test)
# step: 4.2.5
acc = accuracy_score(y_test, y_pred)
acc_list.append(acc)
# step: 4.3
acc_mean = np.mean(acc_list)
env_results[env_name] = round(acc_mean, 3)

```

Step 5 — comparing results: env_results shows 'NB' (Naïve Bayes) achieving the highest accuracy, 0.95, via 4-Fold Cross-Validation:

```

# step: 5
print(env_results)

{'LR': 0.1,
 'DT': 0.85,
 'NB': 0.95,
 'KNN (k3)': 0.85,
 'KNN (k5)': 0.6,
 'ANN (h10)': 0.2,
 'ANN (h100)': 0.65 }

```

Step 6 — building the final model from the chosen environment ('NB'), on the full dataset:

```

# step: 6
model = envs['NB'].fit(X, y)

```

Step 7 — deploying the model, e.g., saving it via the pickle library for use in a software system:

```

# step: 7
import pickle

filename = 'clf.model'
pickle.dump(model, open(filename, 'wb'))

```

Choosing an experimental environment doesn't always mean picking the single highest-scoring one — when results are close, developers can also weigh model complexity, training speed, and deployment-time inference speed, choosing sensibly within the constraints of the system where the model will run.

7.6 Chapter Summary

Good classification analysis starts by confirming the problem predicts a categorical outcome, then whether it's binary or multi-class, and whether class sizes are balanced or skewed — all shaping technique and evaluation choices.

This chapter covered 5 techniques — Logistic Regression, Decision Tree, Naïve Bayes, K-Nearest Neighbor, and Artificial Neural Network — each with different feature-type constraints and parameters. Readers are encouraged to further explore Support Vector Machines, Random Forest, deep learning, and ensemble approaches like Voting.

Evaluation compares actual vs. predicted classes via the confusion matrix (TP, TN, FP, FN), where “Positive” is the wanted class and “Negative” the rest. For balanced binary classes, use Accuracy; for imbalanced binary or multi-class problems, use F1 (from Precision and Recall), or Precision/Recall individually depending on which the problem prioritizes improving.

7.7 Review Questions

1. Give 5 real-world examples of classification analysis problems.
2. For a model predicting digits from handwritten input, discuss whether this is a regression or classification problem, with reasoning.
3. From an example decision tree diagram, write a Python function implementing that model's logic.
4. Discuss the advantages and disadvantages of using multiple hidden layers (a Multi-Layer Perceptron).
5. Given: an AI system sorts genuine gold to be boxed for sale, discarding fakes. Testing on 100 gold pieces found: 30 genuine pieces boxed, 20 genuine pieces discarded, and 10 fake pieces boxed. Compute: (a) TP, (b) TN, (c) FP, (d) FN, (e) Accuracy, (f) Precision, (g) Recall, (h) F1.
6. Study the Titanic survival dataset from Kaggle, referenced at <https://github.com/Rathachai/DALAB/tree/gh-pages/exercises> (“Titanic Survivors Dataset”), and write Python code comparing classification models to predict Survived, explaining each technique, its parameters, and its performance.
7. Study the mushroom classification dataset from Kaggle at the same page (“Mushroom Classification Dataset”), and write Python code comparing classification models to predict mushroom Class, explaining each technique, its parameters, and its performance.
8. Study the drug classification dataset from Kaggle at the same page (“Drug Classification Dataset”), and write Python code comparing classification models to predict Drug type, explaining each technique, its parameters, and its performance.

7.8 References

- Bishop, C. M., & Nasrabadi, N. M. (2006). Pattern Recognition and Machine Learning (Vol. 4, No. 4). Springer.
- Boschetti, A., & Massaron, L. (2015). Python Data Science Essentials. Packt Publishing Ltd.
- Brunton, S. L., & Kutz, J. N. (2022). Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control. Cambridge University Press.
- Igual, L., & Seguí, S. (2017). Introduction to Data Science. Springer, Cham.
- O’Neil, C., & Schutt, R. (2013). Doing Data Science: Straight Talk from the Frontline. O’Reilly Media, Inc.
- Provost, F., & Fawcett, T. (2013). Data Science for Business: What You Need to Know about Data Mining and Data-Analytic Thinking. O’Reilly Media, Inc.
- Russell, S., & Norvig, P. (2002). Artificial Intelligence: A Modern Approach.
- VanderPlas, J. (2016). Python Data Science Handbook: Essential Tools for Working with Data. O’Reilly Media, Inc.